

Advanced Computer Programming

Traccia ricostruita — Esercizi/02_JAVA/03_Java_Networking

Premessa

Nel repository ACP-unina/acp_materiale la cartella Esercizi/02_JAVA/03_Java_Networking/Java-networking contiene solo il progetto Eclipse con i sorgenti già svolti: non è presente alcun PDF di traccia (le tracce vere e proprie stanno sotto Esercitazioni/). Quella che segue è la traccia ricostruita per reverse engineering a partire dal codice dei sei file sorgente, con le specifiche implementative che il codice rende obbligatorie.

Struttura attesa del progetto

```
Java-networking/  
src/  
  client/  
    LetturaDocWeb.java  
    TCPClient.java  
    UDPClient.java  
  server/  
    TCPServer.java  
    WorkerServer.java  
    UDPServer.java
```

Parametri comuni a tutti gli esercizi

Parametro	Valore
Host	localhost (127.0.0.1), ottenuto con <code>InetAddress.getByName()</code>
Porta di servizio	8050 (sia per TCP sia per UDP)
Package	client e server
Tracciamento	ogni fase stampata a video con prefisso [Client], [Server], [Server-Worker]
Gestione errori	blocchi try/catch distinti per <code>MalformedURLException</code> , <code>SocketException</code> , <code>IOException</code> , <code>InterruptedException</code>

Esercizio 1 — Lettura di un documento Web (classe LetturaDocWeb)

Scrivere un'applicazione Java che, utilizzando le classi di alto livello del package `java.net`, scarichi e stampi a video il contenuto HTML di una pagina Web, senza gestire esplicitamente socket né protocollo HTTP.

Requisiti:

- costruire un oggetto URL a partire dalla stringa dell'indirizzo (nell'esempio `http://www.scuolapsb.unina.it/`);
- ottenere la connessione con `openConnection()` e aprirla con `connect()`;
- incapsulare lo stream di ingresso in un `InputStreamReader` e poi in un `BufferedReader`, per leggere il documento riga per riga;

- stampare ogni riga finché `readLine()` non restituisce `null`;
- gestire separatamente `MalformedURLException` (URL sintatticamente errato) e `IOException` (errore di connessione o di lettura).

Obiettivo didattico: mostrare il livello di astrazione più alto del networking in Java, in cui la risorsa remota è vista come un semplice stream di byte.

Esercizio 2 — Client/server TCP concorrente

Realizzare una coppia client/server su protocollo TCP in cui il client invia una stringa di richiesta e il server risponde con una stringa di conferma. Il server deve essere **concorrente**: deve poter servire più client contemporaneamente delegando ogni connessione a un thread dedicato.

2a — Lato server (TCPServer)

- creare una `ServerSocket` specificando porta (8050), lunghezza della coda di backlog (50) e indirizzo di bind ottenuto da `InetAddress`;
- entrare in un ciclo infinito in cui si invoca `accept()`, che blocca il thread fino all'arrivo di una richiesta di connessione;
- per ogni `Socket` restituita da `accept()`, istanziare e avviare un thread `WorkerServer`, tornando subito in attesa di nuove connessioni.

2b — Lato worker (WorkerServer)

- la classe estende `Thread` e riceve nel costruttore la socket di connessione, memorizzandola in un campo privato;
- nel metodo `run()`: ricavare dalla socket un `DataInputStream` e un `DataOutputStream`;
- leggere la stringa del client con `readUTF()` e stamparla;
- inviare la risposta con `writeUTF("richiesta ricevuta")`;
- chiudere ordinatamente i due stream e la socket.

2c — Lato client (TCPClient)

- aprire una `Socket` verso `localhost:8050` (la connessione a tre vie avviene nel costruttore);
- creare `DataOutputStream` e `DataInputStream` sulla socket;
- inviare la stringa "Hello server!" con `writeUTF()`;
- attendere la risposta con `readUTF()` e stamparla;
- chiudere stream e socket.

Prova di funzionamento richiesta: avviare il server, poi lanciare più istanze del client e verificare dai messaggi di log che per ciascuna connessione venga creato un worker distinto e che il server resti sempre disponibile su `accept()`.

Esercizio 3 — Client/server UDP

Realizzare la stessa interazione richiesta/risposta dell'esercizio 2 utilizzando però il protocollo UDP, quindi datagrammi e comunicazione non connessa.

3a — Lato server (UDPServer)

- creare una `DatagramSocket` legata alla porta 8050 e all'indirizzo locale;
- predisporre un buffer di byte di dimensione pari al massimo payload UDP (65508) e un `DatagramPacket` di ricezione;
- invocare `receive()` (chiamata bloccante) e ricostruire la stringa ricevuta dai byte del pacchetto, usando `getLength()` per delimitare la porzione effettivamente occupata;
- costruire il pacchetto di risposta con il testo "OK client, pacchetto ricevuto", ricavando IP e porta del destinatario dal pacchetto ricevuto con `getAddress()` e `getPort()`;
- introdurre un ritardo artificiale di 5 secondi (`Thread.sleep(5000)`) prima dell'invio, per rendere osservabile il blocco del client sulla `receive()`;
- inviare la risposta con `send()` e chiudere la socket.

3b — Lato client (`UDPCClient`)

- creare una `DatagramSocket` senza argomenti, lasciando che il sistema assegni una porta effimera;
- costruire un `DatagramPacket` con i byte della stringa "Hello Server!", la sua lunghezza, l'`InetAddress` del server e la porta 8050;
- inviare il pacchetto con `send()`;
- predisporre un secondo `DatagramPacket` di ricezione sullo stesso buffer da 65508 byte e invocare `receive()`;
- ricostruire e stampare la stringa di risposta, quindi chiudere la socket.

Nota implementativa: la stessa `DatagramSocket` viene usata per inviare e ricevere; non esiste alcuna connessione, quindi l'indirizzamento è interamente a carico del singolo pacchetto.

Domande di approfondimento (tipiche in sede d'esame)

- Perché il server TCP deve delegare a un thread worker invece di servire il client nel ciclo principale? Che cosa accadrebbe altrimenti?
- Che differenza c'è fra il parametro di backlog della `ServerSocket` e il numero di thread worker attivi?
- Perché `writeUTF/readUTF` funzionano correttamente su TCP mentre su UDP occorre gestire manualmente i byte?
- Perché nella ricostruzione della stringa da un datagramma si deve usare `getLength()` e non la lunghezza del buffer?
- Il server UDP dell'esercizio 3 serve un solo client e termina: come lo si renderebbe concorrente e permanente?
- In quali casi conviene UDP rispetto a TCP, alla luce dell'assenza di handshake, ordinamento e ritrasmissione?